

Neural Driving

Neuroevolution of an Autonomous 2D Racing Controller

Explicit neural network, evolutionary optimization and custom vehicle simulation

Author: Alexandre Sigiridis-Schlepp
Background: ENSAM / Karlsruhe Institute of Technology (KIT)
Project type: Personal engineering / AI portfolio project
Language: Python
Main libraries: NumPy, Pygame
Core method: Fixed-topology MLP optimized by neuroevolution
Date: September 2026

Scope statement. This project is a self-contained 2D autonomous-driving experiment. It does not use TensorFlow, PyTorch, automatic differentiation or backpropagation during training. The neural controller is optimized with a genetic algorithm acting directly on all weights and biases. The vehicle dynamics are deliberately simplified and use pixel-second units; the project is intended to study learning, control, numerical implementation and evolutionary optimization rather than to reproduce a road vehicle in SI units.

Abstract

This project develops a complete autonomous-driving learning environment from first principles in Python. A simulated vehicle is equipped with nine range sensors and a speed input. These ten observations feed a fully connected multilayer perceptron that returns steering, throttle and brake commands. Rather than fitting the network to labelled data or computing gradients, a population of controllers is optimized through neuroevolution using tournament selection, elitism, uniform crossover, Gaussian mutation and random immigrants.

The project includes the entire learning stack: vehicle dynamics, track geometry, analytical ray casting, collision detection, curvilinear progress measurement, ordered checkpoints, fitness design, deterministic fixed-step simulation, model serialization, interactive visualization, a network architecture workshop, a champion library and a player-versus-AI race mode. Particular attention is paid to preventing reward exploits: distance travelled without forward progress is not rewarded, checkpoints must be crossed physically and in order, reverse motion is penalized, and completed races are ranked primarily by race time. A later fitness revision additionally penalizes sustained sub-maximal speed.

On the current *Technique fluide* circuit, the sport-physics experiment with 50 vehicles over ten generations improved the winning two-lap time from approximately 15.49 s to 14.45 s, while the number of two-lap finishers increased from 2 to 33 and collisions decreased from 48 to 17. A subsequent six-generation refinement using the speed-deficit fitness reduced the real two-lap time to 14.41 s. The supplied champion was independently replayed headlessly and reproduced its stored two-lap times. The project therefore demonstrates a transparent implementation of neural control and evolutionary learning while also exposing important limitations: training on one fixed track and spawn point, high sample cost, sensitivity to fitness shaping, and the absence of a physical tyre model or cross-track generalization study.

Contents

Abstract	i
1 Introduction	1
1.1 Motivation	1
1.2 Precise learning terminology	1
1.3 Project objectives	2
2 Software Architecture	3
2.1 Code organization	3
2.2 Operating modes	3
3 Vehicle Dynamics	5
3.1 Units and modelling philosophy	5
3.2 Kinematic baseline	5
3.3 Inertial arcade model	5
3.4 Lateral-velocity relaxation	6
3.5 Speed-dependent steering authority	6
4 Track Geometry, Sensing and Collision Detection	8
4.1 Technical track	8
4.2 Sensor model	8
4.3 Collision model	9
4.4 Ordered checkpoints and progress robustness	9
5 Neural Controller	10
5.1 Inputs and outputs	10
5.2 MLP architecture	10
5.3 Initialization	11
5.4 Alternative network architectures	11
6 Neuroevolution Algorithm	12

6.1	Genome representation	12
6.2	Population evaluation	12
6.3	Selection, crossover and mutation	12
6.4	Algorithm summary	13
6.5	Why no backpropagation is required	13
7	Fitness Design	14
7.1	Why fitness shaping is necessary	14
7.2	Progress of unfinished trajectories	14
7.3	Speed-deficit penalty	14
7.4	Completed races	15
8	Implementation and Reproducibility	16
8.1	Deterministic simulation	16
8.2	Saving models	16
8.3	Interactive explainability	16
9	Verification and Validation	18
9.1	Testing strategy	18
9.2	Independent champion replay	18
10	Results	19
10.1	Sport-physics training experiment	19
10.2	Speed-deficit refinement	21
10.3	Observed network behaviour	22
10.4	Human-versus-AI mode	22
11	Discussion	24
11.1	What the project demonstrates	24
11.2	Important limitations	24
11.3	Why the project is not NEAT	25
12	Recommended Extensions	26
12.1	Generalization benchmark	26
12.2	Multiple-seed statistical study	26
12.3	Controller-state extensions	26
12.4	Optimization comparisons	26
12.5	Vehicle-model extension	27

13 Conclusion	28
A Current Default Parameters	29
B Suggested Final Portfolio Screenshots	30
C Background References	31

List of Figures

2.1	Interactive project menu. Training settings, physics, track, saved champions and direct race modes can be selected without modifying code.	4
3.1	Manual visualization of the inertial vehicle model. The sensor fan provides the controller observations; the motion direction can differ from body heading.	7
4.1	Technical track in manual mode. The cyan rays are the nine distance sensors used by the neural controller.	9
5.1	Dynamic visualization of the neural controller. Edge thickness represents $ w $, edge colour represents the sign of the weight and node colour represents activation. . . .	11
8.1	Interactive explanatory overlay. The project was designed not only to train a controller but also to expose the mechanisms being optimized.	17
10.1	Best and mean fitness during the principal Sport experiment. Elite preservation makes the best-so-far result non-decreasing, while mean fitness can fluctuate. . . .	20
10.2	Population-level robustness. The number of successful two-lap trajectories increases while collisions decrease.	20
10.3	Winning two-lap time and best single lap observed during the Sport experiment. .	21
10.4	Real race time and evaluated time after addition of the speed-deficit penalty. The previous champion remains competitive; a small improvement appears in generation 5. .	21
10.5	Training visualization on the oval/technical environment. Population statistics, current leader, sensor rays and neural architecture are visible during optimization. .	22
10.6	Human-versus-champion race mode. This mode is a demonstration and comparison tool; it is not part of the evolutionary fitness.	23

List of Tables

2.1 Main software modules. 3

6.1 Evolutionary hyperparameters used by the current default configuration. 12

10.1 Evolution of selected metrics in the 10-generation Sport experiment. 19

A.1 Selected parameters from the current `Config` object. 29

Chapter 1

Introduction

1.1 Motivation

Autonomous control is often presented through high-level reinforcement-learning libraries in which the simulator, neural network and optimizer are largely hidden behind an API. The objective of this project was the opposite: implement the complete chain explicitly enough that every important quantity can be inspected and modified.

The project therefore combines four themes that are usually treated separately:

- numerical simulation of a moving rigid body on a closed track;
- geometric perception through analytical range sensors;
- nonlinear control with a small feed-forward neural network;
- black-box optimization of network parameters by an evolutionary algorithm.

The result is a compact environment in which a controller can be evolved, visualized, saved, replayed and challenged by a human driver.

1.2 Precise learning terminology

The project is sometimes informally described as “reinforcement learning” because controllers receive no labelled steering targets and are selected only from a scalar performance signal. Strictly, however, the implemented training algorithm is better described as **neuroevolution**. No value function, policy gradient, temporal-difference target or backpropagation step is computed. A complete neural policy is evaluated over an episode, assigned a fitness, and its parameters are modified by selection, crossover and mutation.

This distinction matters technically. If the neural-network parameters are grouped into a vector

$$\boldsymbol{\theta} \in \mathbb{R}^{N_{\theta}}, \quad (1.1)$$

the implemented optimizer never evaluates

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}). \quad (1.2)$$

It only observes the scalar fitness $J(\boldsymbol{\theta})$ after simulation and generates new candidate vectors stochastically.

1.3 Project objectives

The main engineering objectives were:

1. create a deterministic and reproducible 2D driving simulator;
2. implement vehicle control without a pre-programmed racing line;
3. restrict the neural network to locally measurable information;
4. evolve network weights without a deep-learning framework;
5. design a fitness that rewards actual race progress while limiting trivial exploits;
6. validate geometry, physics, sensing, checkpoints, serialization and evolutionary reproducibility with automated tests;
7. expose the complete process in an interactive interface suitable for experimentation and demonstration.

Chapter 2

Software Architecture

2.1 Code organization

The project is separated by physical or algorithmic responsibility rather than concentrated in one game loop. Table 2.1 summarizes the principal modules.

Table 2.1: Main software modules.

File	Responsibility
<code>main.py</code>	Command-line interface, simulation loop, events and operating modes.
<code>menu.py</code>	Graphical menu, experiment settings, model selection and network workshop.
<code>race.py</code>	Human-versus-champion countdown, lap management and race result.
<code>config.py</code>	Vehicle, network and evolutionary hyperparameters.
<code>car.py</code>	Vehicle dynamics, sensing, collision handling, progress and fitness.
<code>track.py</code>	Track geometry, curvilinear projection, boundaries and checkpoints.
<code>sensors.py</code>	Analytical ray/segment and ray/circle intersection calculations.
<code>neural_network.py</code>	Explicit MLP, forward pass, genome conversion and serialization.
<code>evolution.py</code>	Population evaluation, tournament selection, crossover, mutation and elitism.
<code>visualization.py</code>	Track, cars, sensor rays, statistics and network rendering.

2.2 Operating modes

The same physical and control code is reused in four main modes:

- **manual**: keyboard control, useful for checking drivability and geometry;
- **train**: simultaneous evaluation of a population of neural controllers;
- **champion**: replay of a saved neural controller;
- **race**: player-versus-AI comparison using the same vehicle physics for both cars.

The code also supports a headless training/replay mode, allowing the simulation to run without importing or rendering Pygame graphics.

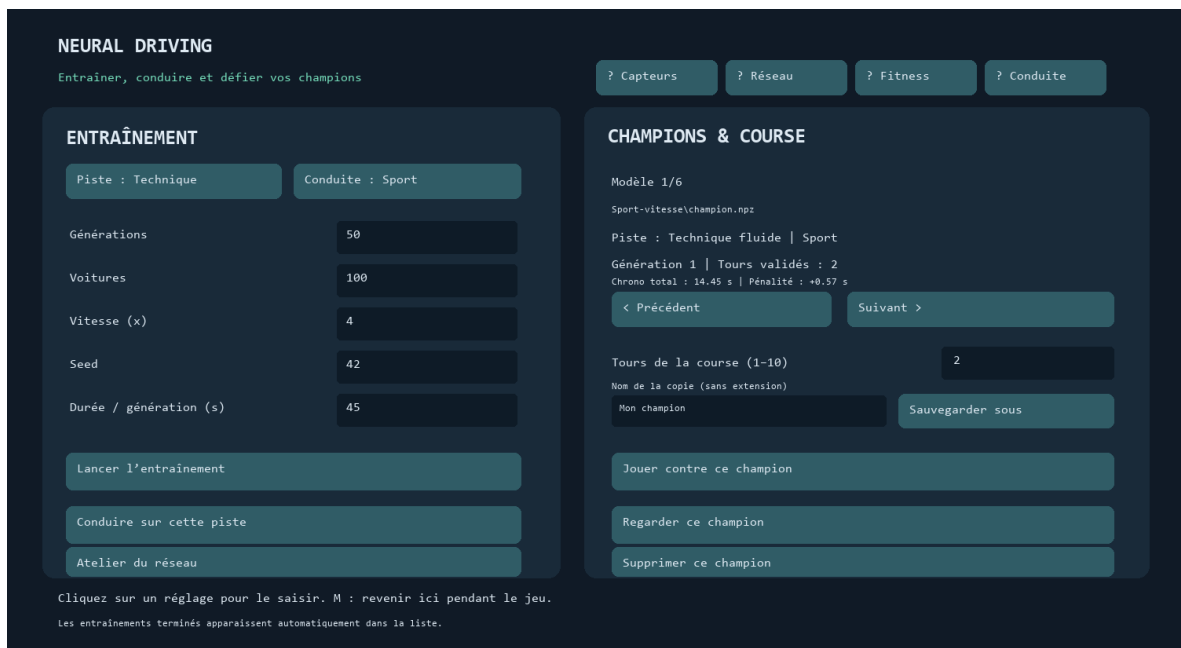


Figure 2.1: Interactive project menu. Training settings, physics, track, saved champions and direct race modes can be selected without modifying code.

Chapter 3

Vehicle Dynamics

3.1 Units and modelling philosophy

The simulated vehicle uses pixels, seconds and radians. Consequently, its parameters should not be interpreted as calibrated automotive quantities. The goal is a controllable dynamic system with meaningful inertia, steering limits and collision behaviour, not a full four-wheel model.

The current configuration uses

$$\Delta t = \frac{1}{30} \text{ s} \quad (3.1)$$

for the logical controller update. The physical step is subdivided further so that collision checks occur roughly every two pixels even near the maximum speed.

3.2 Kinematic baseline

The original baseline is a conventional bicycle-like kinematic model. With position (x, y) , heading ψ , speed v , wheelbase L and steering angle δ ,

$$\dot{x} = v \cos \psi, \quad (3.2)$$

$$\dot{y} = v \sin \psi, \quad (3.3)$$

$$\dot{\psi} = \frac{v}{L} \tan \delta. \quad (3.4)$$

The longitudinal acceleration is represented by a simple actuator/drag balance,

$$\dot{v} = a_e u_t - a_b u_b - c_r v - c_a v^2, \quad (3.5)$$

where $u_t \in [0, 1]$ is throttle and $u_b \in [0, 1]$ is brake.

For the current Sport profile,

$$v_{\max} = 270 \text{ px/s}, \quad a_e = 210 \text{ px/s}^2, \quad a_b = 340 \text{ px/s}^2, \quad (3.6)$$

with $c_r = 0.25$ and $c_a = 0.0015$ in the internal pixel-second unit system.

3.3 Inertial arcade model

The current `inertial-v2` mode separates vehicle orientation from the velocity vector. This prevents the direction of motion from rotating instantaneously when the body turns.

Let $\mathbf{v}$ be the translational velocity, $\mathbf{e}_x = [\cos \psi, \sin \psi]^T$ the vehicle forward direction and $\mathbf{e}_y = [-\sin \psi, \cos \psi]^T$ the lateral direction. Engine force is applied along $\mathbf{e}_x$, while braking and drag reduce the magnitude $\|\mathbf{v}\|$ without creating reverse motion.

A bicycle-model yaw-rate request is first computed from the longitudinal component

$$v_x = \max(0, \mathbf{v} \cdot \mathbf{e}_x), \quad (3.7)$$

then

$$r_{\text{req}} = \frac{v_x}{L} \tan \delta. \quad (3.8)$$

To avoid arbitrarily high lateral acceleration, the request is limited according to

$$|r_{\text{req}}| \leq \frac{a_{y,\text{max}}}{\max(v_x, 25)}. \quad (3.9)$$

Yaw rate follows this request through a first-order response,

$$\dot{r} \approx \frac{r_{\text{req}} - r}{\tau_r}, \quad (3.10)$$

implemented exactly over a numerical step through the exponential relaxation factor

$$r_{k+1} = r_k + (r_{\text{req}} - r_k) \left(1 - e^{-h/\tau_r}\right). \quad (3.11)$$

For Sport physics, $\tau_r = 0.055$ s.

3.4 Lateral-velocity relaxation

The lateral velocity

$$v_y = \mathbf{v} \cdot \mathbf{e}_y \quad (3.12)$$

is gradually dissipated instead of cancelled instantaneously. The desired correction over a substep h is

$$\Delta v_y^* = -v_y \left(1 - e^{-\lambda_y h}\right), \quad (3.13)$$

then clipped by the available lateral acceleration,

$$|\Delta v_y| \leq a_{y,\text{max}} h. \quad (3.14)$$

The current Sport values are $\lambda_y = 32 \text{ s}^{-1}$ and $a_{y,\text{max}} = 1100 \text{ px/s}^2$.

The resulting slip angle is monitored as

$$\beta = \text{wrap}_{[-\pi, \pi]} \left(\text{atan2}(v_y^{\text{global}}, v_x^{\text{global}}) - \psi \right). \quad (3.15)$$

This is a diagnostic angle only; the model does not contain a tyre slip-force law.

3.5 Speed-dependent steering authority

The steering limit decreases with speed,

$$\delta_{\text{max}}(v) = \frac{\delta_0}{1 + k_s (v/v_{\text{max}})^2}, \quad (3.16)$$

with $\delta_0 = 0.52$ rad and $k_s = 0.35$ for the Sport profile. Steering angle itself is rate-limited to 4 rad/s.

This simple mechanism has a useful control consequence: a network cannot request an unrealistically abrupt high-speed turn merely by saturating its steering output.

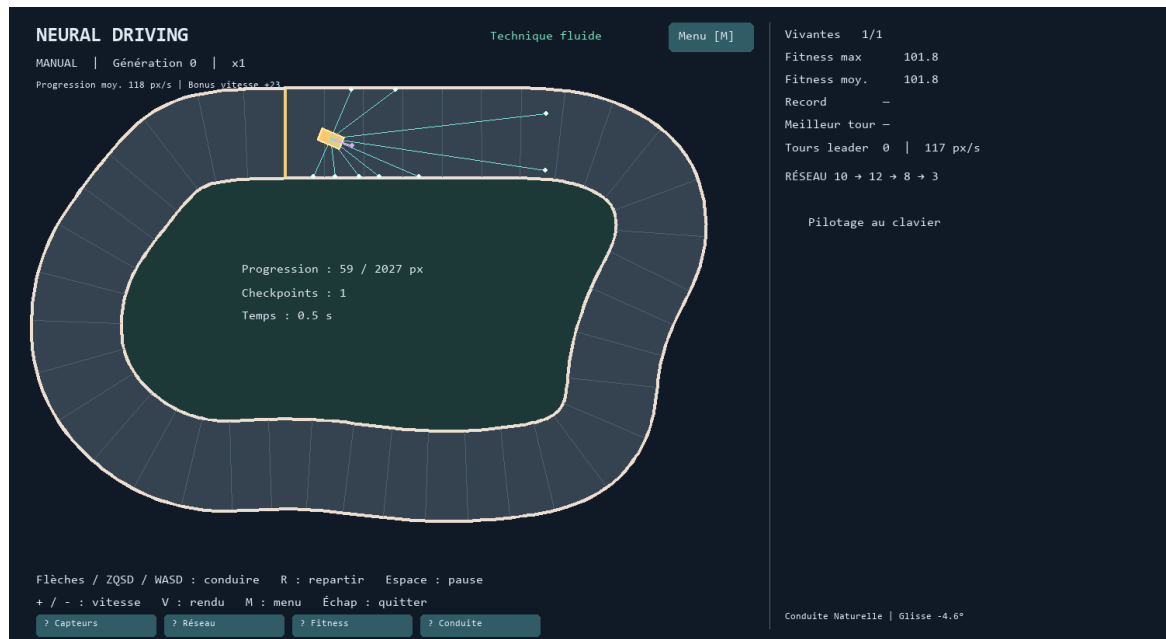


Figure 3.1: Manual visualization of the inertial vehicle model. The sensor fan provides the controller observations; the motion direction can differ from body heading.

Chapter 4

Track Geometry, Sensing and Collision Detection

4.1 Technical track

The current training track, *Technique fluide* (`technical-v2`), is a closed polygonal centreline constructed from a smoothed technical layout blended toward an oval reference. The resulting centreline is approximately 2027 px long, the road is 112 px wide and the track contains 40 ordered checkpoint gates.

A curvilinear coordinate $s \in [0, L_{\text{track}})$ is defined by projection onto successive centreline segments. This scalar coordinate is central to progress measurement, but checkpoint validation is deliberately not based only on s because projection can jump between adjacent segments in tight curvature.

4.2 Sensor model

Nine range sensors are emitted at local angular offsets

$$\{-90, -60, -30, -15, 0, 15, 30, 60, 90\}^\circ. \quad (4.1)$$

Each ray has maximum range $R = 280$ px. For a ray

$$\mathbf{p}(t) = \mathbf{p}_0 + t\mathbf{d}, \quad t \geq 0, \quad (4.2)$$

and a wall segment

$$\mathbf{q}(u) = \mathbf{a} + u\mathbf{e}, \quad 0 \leq u \leq 1, \quad (4.3)$$

the two-dimensional cross product gives the intersection parameters

$$t = \frac{(\mathbf{a} - \mathbf{p}_0) \times \mathbf{e}}{\mathbf{d} \times \mathbf{e}}, \quad (4.4)$$

$$u = \frac{(\mathbf{a} - \mathbf{p}_0) \times \mathbf{d}}{\mathbf{d} \times \mathbf{e}}. \quad (4.5)$$

Among valid intersections, the smallest non-negative t is retained and clipped at R . The neural input is the normalized distance

$$x_i = \frac{\min(t_i, R)}{R} \in [0, 1]. \quad (4.6)$$

Hence, $x_i = 1$ means no wall was detected within sensor range.

4.3 Collision model

The vehicle is represented by a rectangle of length 30 px and width 16 px. A collision is detected using the complete transformed rectangle rather than only its centre point. On the technical track, corner inclusion and explicit vehicle-edge/wall-segment intersections are checked. This avoids the common failure mode in which the centre remains on the track while one corner cuts through the inside boundary.

The motion step is subdivided according to maximum speed. If the logical time step is Δt , the number of physical substeps is approximately

$$N_{\text{sub}} = \left\lceil \Delta t \max \left(90, \frac{v_{\text{max}}}{2} \right) \right\rceil. \quad (4.7)$$

With $v_{\text{max}} = 270$ px/s and $\Delta t = 1/30$ s, this results in several collision checks per control step and prevents a fast vehicle from jumping across a thin boundary in one update.

4.4 Ordered checkpoints and progress robustness

A checkpoint is counted only when the physical segment swept by the car crosses the next gate in the positive direction. The crossing point must lie between both gate endpoints. Thus, a sudden numerical jump in centreline projection cannot independently validate a checkpoint.

Forward progress is based on the historical maximum of a bounded curvilinear coordinate. Reverse travel is accumulated separately and penalized. This structure was introduced after a regression case in which a 0.01 px physical displacement created a roughly 2.1 px projection jump between nearby polyline segments. The current implementation clips such projection anomalies and waits for the next physical gate crossing to re-synchronize the progress state.

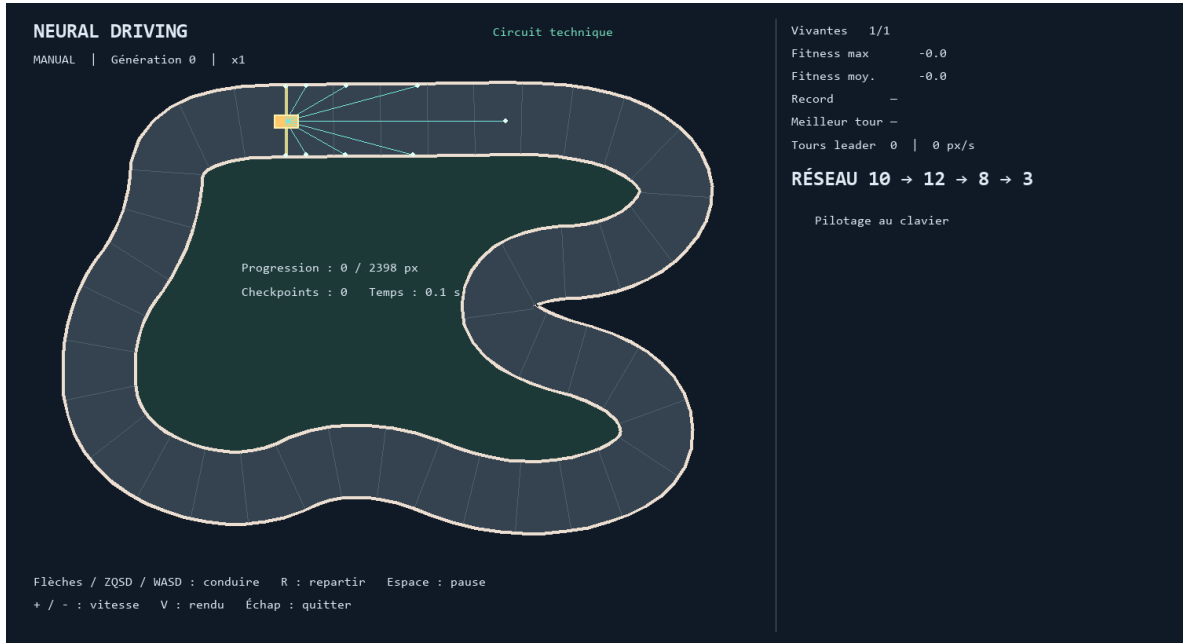


Figure 4.1: Technical track in manual mode. The cyan rays are the nine distance sensors used by the neural controller.

Chapter 5

Neural Controller

5.1 Inputs and outputs

The controller is intentionally given no global racing-line information. Its ten inputs are

$$\mathbf{x} = \begin{bmatrix} d_{-90}/R & d_{-60}/R & \cdots & d_{+90}/R & v/v_{\max} \end{bmatrix}^T \in \mathbb{R}^{10}. \quad (5.1)$$

There is no absolute (x, y) position, checkpoint index, centreline tangent or hidden target direction in the observation vector.

The three outputs are

$$\mathbf{u} = \begin{bmatrix} u_s & u_t & u_b \end{bmatrix}^T, \quad (5.2)$$

where steering $u_s \in [-1, 1]$ and throttle/brake $u_t, u_b \in [0, 1]$.

5.2 MLP architecture

The default network is

$$\boxed{10 \rightarrow 12 \rightarrow 8 \rightarrow 3}. \quad (5.3)$$

For a hidden layer ℓ ,

$$\mathbf{a}^{(\ell+1)} = f\left(W^{(\ell)}\mathbf{a}^{(\ell)} + \mathbf{b}^{(\ell)}\right). \quad (5.4)$$

With the default `tanh` hidden activation,

$$\mathbf{h}_1 = \tanh(W_0\mathbf{x} + \mathbf{b}_0), \quad (5.5)$$

$$\mathbf{h}_2 = \tanh(W_1\mathbf{h}_1 + \mathbf{b}_1), \quad (5.6)$$

$$\mathbf{z} = W_2\mathbf{h}_2 + \mathbf{b}_2. \quad (5.7)$$

The physical output mappings are

$$u_s = \tanh(z_0), \quad (5.8)$$

$$u_t = \sigma(z_1), \quad (5.9)$$

$$u_b = \sigma(z_2), \quad (5.10)$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (5.11)$$

The matrix and bias dimensions are

$$W_0 \in \mathbb{R}^{12 \times 10}, \quad \mathbf{b}_0 \in \mathbb{R}^{12}, \quad (5.12)$$

$$W_1 \in \mathbb{R}^{8 \times 12}, \quad \mathbf{b}_1 \in \mathbb{R}^8, \quad (5.13)$$

$$W_2 \in \mathbb{R}^{3 \times 8}, \quad \mathbf{b}_2 \in \mathbb{R}^3. \quad (5.14)$$

Therefore the total number of optimized scalar parameters is

$$N_\theta = 12 \cdot 10 + 12 + 8 \cdot 12 + 8 + 3 \cdot 8 + 3 = 263. \quad (5.15)$$

5.3 Initialization

Weights are initialized from zero-mean Gaussian distributions with a variance scaled according to adjacent layer sizes,

$$W_{ij}^{(\ell)} \sim \mathcal{N}\left(0, \frac{2}{n_\ell + n_{\ell+1}}\right). \quad (5.16)$$

Hidden biases start at zero. The two output biases associated with throttle and brake are initialized to +1 and -2, respectively. This is a minimal mechanical prior that encourages initial networks to attempt forward motion; it does not encode a steering trajectory.

5.4 Alternative network architectures

The interactive network workshop permits one to three hidden layers, each containing 2–24 neurons, with `tanh`, ReLU or sigmoid hidden activation. The network topology is selected before an experiment and then remains fixed during that evolutionary run. This is therefore not NEAT: evolution acts on parameters, not topology.

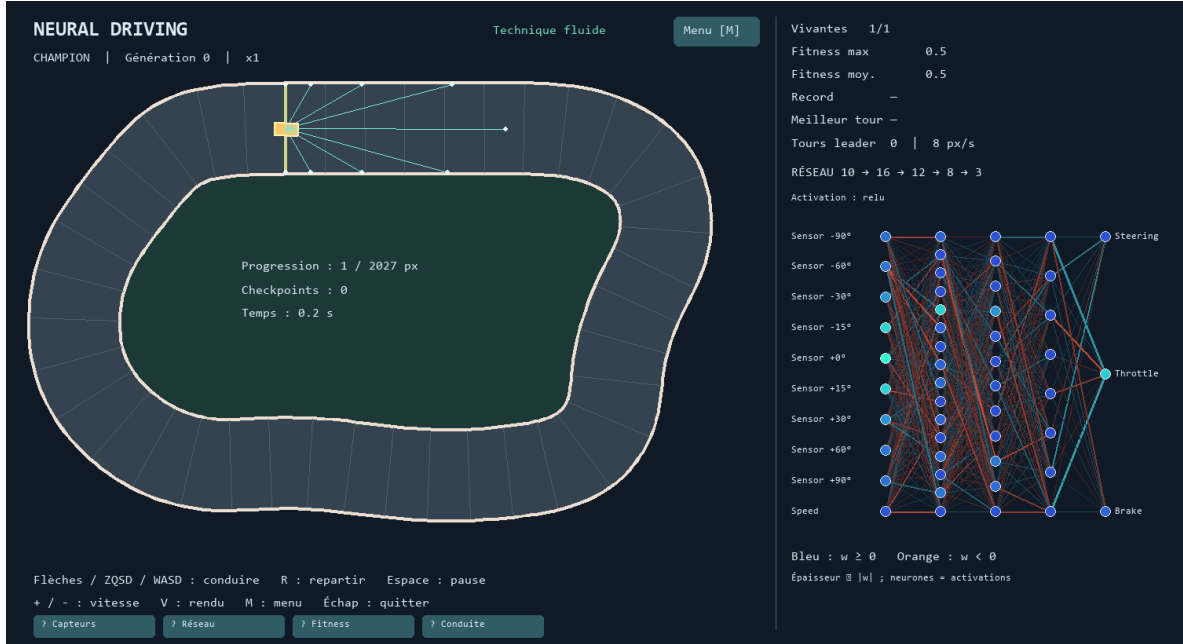


Figure 5.1: Dynamic visualization of the neural controller. Edge thickness represents $|w|$, edge colour represents the sign of the weight and node colour represents activation.

Chapter 6

Neuroevolution Algorithm

6.1 Genome representation

All weight matrices and bias vectors are flattened and concatenated into one genome,

$$\mathbf{g} = \left[\text{vec}(W_0) \quad \text{vec}(W_1) \quad \text{vec}(W_2) \quad \mathbf{b}_0 \quad \mathbf{b}_1 \quad \mathbf{b}_2 \right]^T. \quad (6.1)$$

For the default controller, $\mathbf{g} \in \mathbb{R}^{263}$.

6.2 Population evaluation

Every controller starts from the same initial position and is simulated independently. Vehicle-to-vehicle collisions are intentionally disabled during training; otherwise one controller’s score would depend on other members of the same generation.

A run terminates for an individual after collision, prolonged lack of progress, the episode time limit or completion of the requested number of laps. After all cars are inactive, fitness values are ranked and the next generation is produced.

6.3 Selection, crossover and mutation

The current default evolutionary parameters are listed in Table 6.1.

Table 6.1: Evolutionary hyperparameters used by the current default configuration.

Parameter	Value
Population	100 (50 in the principal Sport experiment)
Tournament size	5
Elite count	5
Random immigrants	5
Crossover probability	0.35
Per-parameter mutation probability	0.20
Mutation standard deviation	0.25
Genome clipping interval	$[-8, 8]$
Default seed	42

Tournament selection samples five population indices uniformly and retains the highest-fitness candidate as a parent. The best five networks are copied unchanged to the new generation. For non-elite offspring, uniform crossover is used with probability 0.35:

$$g_i^{\text{child}} = \begin{cases} g_i^{(A)}, & r_i < 0.5, \\ g_i^{(B)}, & r_i \geq 0.5. \end{cases} \quad (6.2)$$

Each parameter then has probability $p_m = 0.2$ of Gaussian mutation,

$$g'_i = g_i + m_i \epsilon_i, \quad m_i \sim \text{Bernoulli}(0.2), \quad \epsilon_i \sim \mathcal{N}(0, 0.25^2). \quad (6.3)$$

Finally, parameters are clipped to $[-8, 8]$. Five completely new random networks are added each generation to maintain diversity.

6.4 Algorithm summary

Listing 6.1: High-level neuroevolution loop.

```
initialize P neural networks
for generation = 1 ... G:
    reset one simulated car per network
    while at least one car is alive:
        sense track distances and normalized speed
        compute steering, throttle, brake = network.forward(inputs)
        integrate vehicle dynamics and collisions
        update progress, checkpoints and fitness terms
    rank networks by final fitness
    save champion if record improves
    copy elites unchanged
    fill population using tournament parents:
        optional uniform crossover
        Gaussian mutation of weights and biases
    add random immigrants
```

6.5 Why no backpropagation is required

The MLP forward pass is differentiable, but the complete simulator contains discontinuous events: wall collisions, checkpoint crossings, episode termination and rank-based reproduction. Neuroevolution avoids differentiating through any of these operations. It treats the complete simulator as a black-box map

$$\theta \mapsto J(\theta). \quad (6.4)$$

The price is sample efficiency: many full episodes are needed because the optimizer has no local gradient information.

Chapter 7

Fitness Design

7.1 Why fitness shaping is necessary

A naive reward such as travelled Euclidean distance can be exploited by driving in circles. A naive centreline projection can jump between nearby branches. Rewarding speed alone encourages accelerating into a wall. The fitness therefore combines several terms while keeping completed races strictly above incomplete trajectories.

7.2 Progress of unfinished trajectories

For a vehicle that has not completed the required number of laps, the current fitness is

$$J_{\text{partial}} = s_{\text{max}} + 20N_{\text{cp}} + B_v - 0.1D_{\text{reverse}} - 0.2t - P_{\text{stop}} - 100P_{\text{speed}}, \quad (7.1)$$

clipped below 10^6 . Here s_{max} is maximum validated progress, N_{cp} is the ordered checkpoint count, and D_{reverse} is accumulated backwards curvilinear motion.

The progress-speed bonus is

$$\bar{v}_p = \frac{s_{\text{max}}}{\max(t, \Delta t)}, \quad (7.2)$$

$$B_v = 0.6s_{\text{max}} \min\left(\frac{\bar{v}_p}{v_{\text{max}}}, 1\right). \quad (7.3)$$

This rewards achieving the same progress in less time, not merely maintaining a high instantaneous vehicle speed.

7.3 Speed-deficit penalty

The newest fitness revision integrates the normalized speed deficit,

$$D_v = \int_0^T \left(1 - \frac{v(t)}{v_{\text{max}}}\right) dt, \quad (7.4)$$

implemented numerically with trapezoidal averaging over each physical substep. The associated time penalty is

$$P_{\text{speed}} = w_v D_v, \quad (7.5)$$

with $w_v = 0.5$ in the current configuration.

Thus one second at zero speed contributes 0.5 s to the evaluated time, one second at half speed contributes 0.25 s, and operation at $v_{\max}$ contributes zero.

7.4 Completed races

A completed race always ranks above an incomplete trajectory. For a finisher,

$$J_{\text{finish}} = 10^6 + \frac{10^6}{1 + T_{\text{race}} + P_{\text{speed}}}. \quad (7.6)$$

Consequently, among finishers, the objective is effectively to minimize an evaluated race time. This avoids a slower complete trajectory winning because of extra progress or checkpoint bonuses.

Chapter 8

Implementation and Reproducibility

8.1 Deterministic simulation

The simulator uses a fixed logical time step and a dedicated NumPy random generator initialized from a configurable seed. At unchanged code, dependencies, seed, experiment settings and initial model, the recorded validation found reproducible generation statistics independent of graphical rendering.

8.2 Saving models

A saved champion contains matrices, biases and JSON metadata in a compressed NumPy archive. Metadata records network layers, activation, track, physics configuration, generation, fitness and lap times. Statistics are also written to CSV after each completed generation, while the run configuration is saved separately in JSON.

This separation permits three levels of reproducibility:

- `config.json`: reproduce experiment parameters;
- `stats.csv`: audit generation-level outcomes;
- `champion.npz`: replay the best policy independently.

8.3 Interactive explainability

The program can display the leader's sensor rays, network architecture, signed weights and activations in real time. It also contains educational overlays explaining sensors, the network, fitness and vehicle control.



Figure 8.1: Interactive explanatory overlay. The project was designed not only to train a controller but also to expose the mechanisms being optimized.

Chapter 9

Verification and Validation

9.1 Testing strategy

The project contains automated regression tests for the physical engine, geometry, sensing, learning logic, serialization and graphical interactions. The supplied validation log records 37 passing tests for the final speed-penalty revision on the original Windows/Python 3.12 validation environment.

The principal checks include:

- acceleration, braking, steering response and inertial yaw dynamics;
- bounded lateral grip and decay of lateral slip;
- substep stability and high-speed collision detection;
- exact or independently sampled sensor distances;
- centreline projection and track seam consistency;
- complete-rectangle collision checks around the entire track;
- physical checkpoint crossing and two-lap counting;
- prevention of numerical projection jumps from validating checkpoints;
- network output bounds and exact save/load round-trip;
- random-seed reproduction and preservation of elites;
- menu, race, model-management and visualization behaviour.

A geometric test driver exists only inside the test suite to verify that the track and checkpoint system are traversable. It is not used during network training.

9.2 Independent champion replay

The current `Sport-vitesse/champion.npz` file can be replayed headlessly. The supplied project reproduces

$$T_1 = 7.7333 \text{ s}, \quad T_2 = 6.6733 \text{ s}, \quad (9.1)$$

therefore

$$T_{\text{race}} = 14.4067 \text{ s}, \quad (9.2)$$

with two validated laps and termination reason **finished**. This check is important because it validates the serialization/reload path, not only the in-memory training state.

Chapter 10

Results

10.1 Sport-physics training experiment

A principal experiment used the *Technique fluide* track, `inertial-v2` Sport physics, a $10 \rightarrow 12 \rightarrow 8 \rightarrow 3$ tanh network, a population of 50 and seed 42 for ten generations.

The initial generation already contained some successful two-lap controllers, so this experiment should be interpreted as *performance refinement and population improvement*, not as proof that a car learned a complete lap from zero capability in ten generations.

Table 10.1: Evolution of selected metrics in the 10-generation Sport experiment.

Metric	Generation 1	Generation 10
Best fitness	1,060,655	1,064,711
Mean fitness	42,320	701,376
Two-lap finishers / 50	2	33
Collisions / 50	48	17
Winning two-lap time	15.4867 s	14.4533 s
Best lap observed	7.2600 s	6.7333 s
Winner progress speed	261.8 px/s	280.5 px/s

The strongest signal is not only the improvement of the single champion but the shift of the population: finishers increased from 4% to 66% while collisions fell from 96% to 34%.

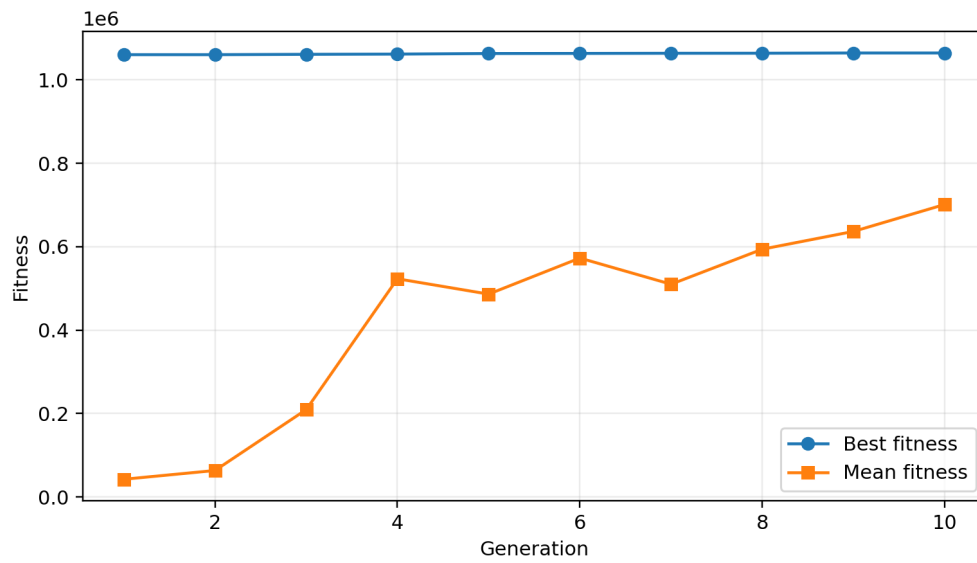


Figure 10.1: Best and mean fitness during the principal Sport experiment. Elite preservation makes the best-so-far result non-decreasing, while mean fitness can fluctuate.

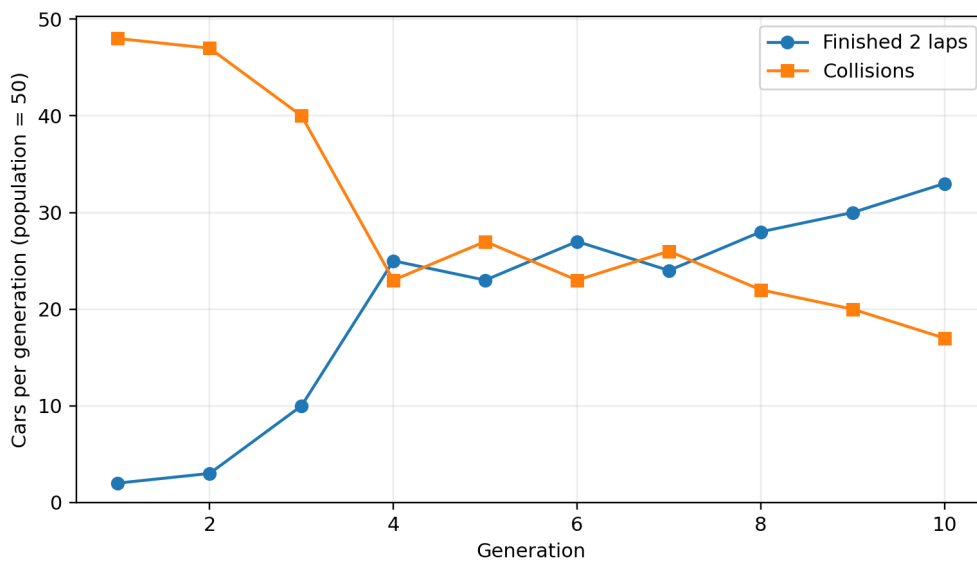


Figure 10.2: Population-level robustness. The number of successful two-lap trajectories increases while collisions decrease.

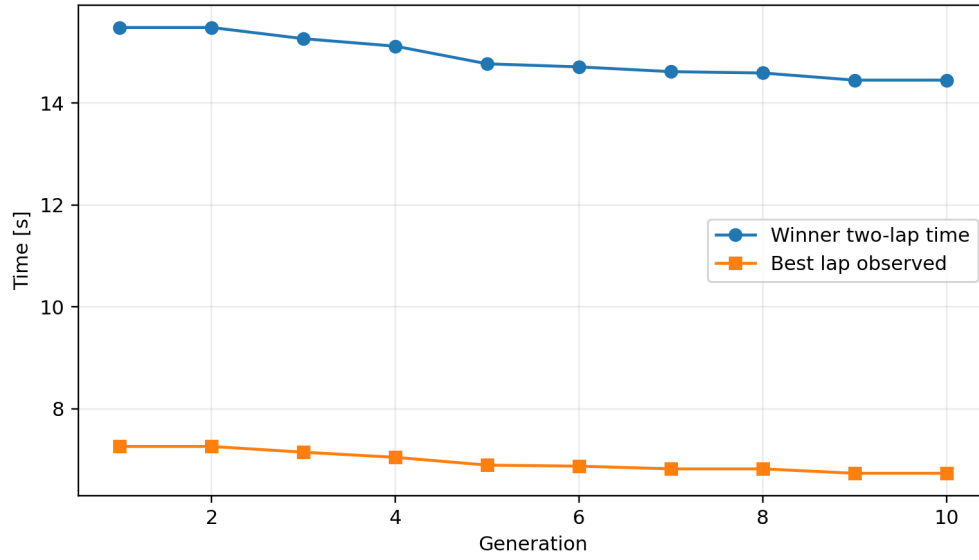


Figure 10.3: Winning two-lap time and best single lap observed during the Sport experiment.

10.2 Speed-deficit refinement

The later `race-speed-v4` fitness was tested for six generations with a population of 40 and the previous Sport champion used as an initial individual. The best real two-lap time improved from 14.4533 s to 14.4067 s. The final controller accumulated a speed-deficit integral of approximately 1.1814 s, corresponding to a weighted penalty of 0.5907 s.

The improvement is modest and should be reported as such. The main value of this experiment is methodological: it verifies that a race-time objective can be augmented with a continuous measure of lost speed without changing the strict priority given to completed races.

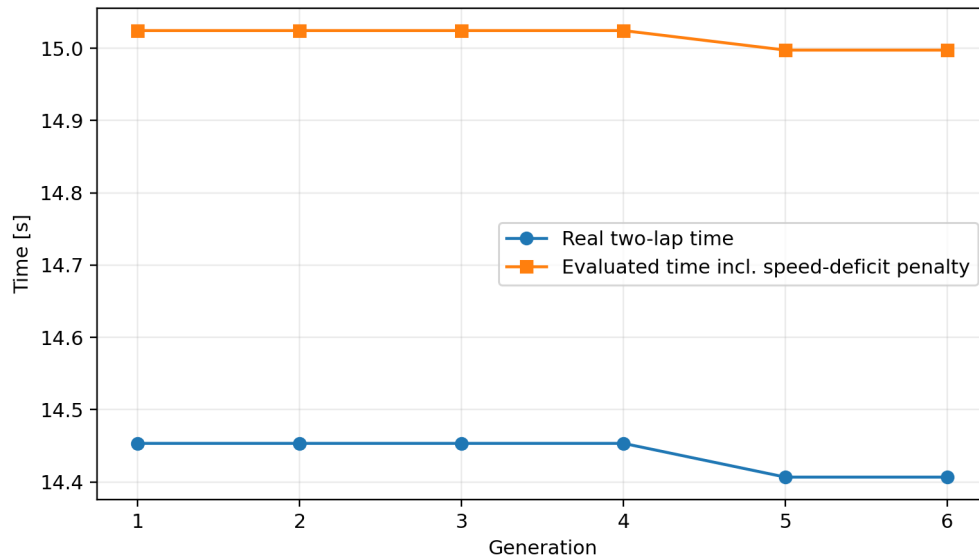


Figure 10.4: Real race time and evaluated time after addition of the speed-deficit penalty. The previous champion remains competitive; a small improvement appears in generation 5.

10.3 Observed network behaviour

The controller is memoryless: at each step, its command is a deterministic nonlinear function of the current nine sensor ranges and speed. Nevertheless, because the vehicle state itself contains inertia, identical sensor distances can produce different future trajectories depending on velocity direction and angular velocity. These hidden physical states are not directly provided to the network, which is one source of partial observability.

The visualization makes the policy inspectable at the level of current neural activations but does not by itself provide a causal interpretation of individual weights. A useful next analysis would compare input-output sensitivities along a full lap or perturb individual sensor values around representative states.

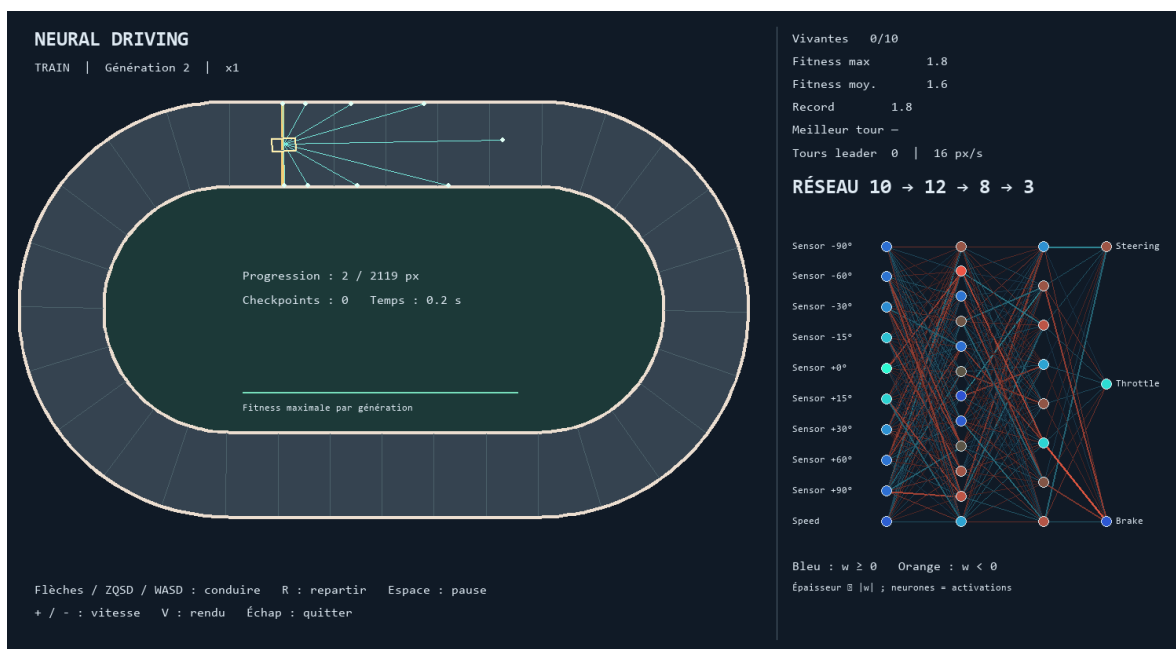


Figure 10.5: Training visualization on the oval/technical environment. Population statistics, current leader, sensor rays and neural architecture are visible during optimization.

10.4 Human-versus-AI mode

A saved champion can be challenged directly. Player and AI use the same selected vehicle physics, start after the same countdown and are treated as ghost vehicles with respect to each other so that collisions between them cannot alter the comparison.

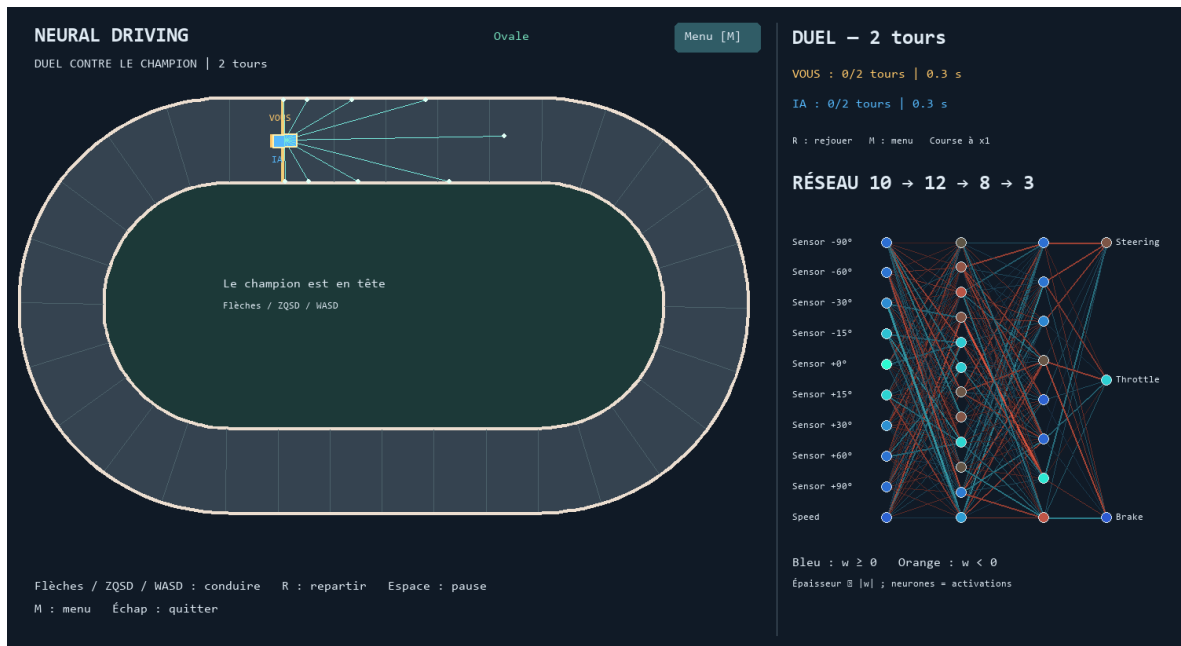


Figure 10.6: Human-versus-champion race mode. This mode is a demonstration and comparison tool; it is not part of the evolutionary fitness.

Chapter 11

Discussion

11.1 What the project demonstrates

The strongest aspect of the project is not the absolute lap time but the completeness and transparency of the engineering chain. The implementation contains no hidden training framework: sensor geometry, network matrices, genome encoding, selection, mutation, physics, collision checks and fitness are all explicit.

This makes the project useful as a compact study of several engineering concepts:

- nonlinear state feedback with a neural controller;
- discretization and numerical stability in a dynamic simulation;
- geometric algorithms for ray casting and collision detection;
- objective-function design and unintended optimization incentives;
- stochastic population optimization and elitist selection;
- experimental reproducibility through deterministic seeds and saved metadata.

11.2 Important limitations

The current results must not be interpreted as evidence of general autonomous-driving capability. The main limitations are:

1. **Single training environment.** Most optimization occurs on one fixed circuit and one fixed spawn position. A controller may exploit track-specific sensor patterns.
2. **No held-out generalization benchmark.** There is no systematic train/validation/test split across unseen circuits.
3. **Simplified vehicle dynamics.** Tyre forces, load transfer, suspension, torque limits and SI-unit calibration are absent.
4. **Memoryless policy.** The MLP does not directly observe yaw rate, slip angle or previous commands and has no recurrent state.
5. **Fitness sensitivity.** Evolution optimizes exactly what is scored. Small changes in time, speed, progress or collision penalties can alter the selected driving style.
6. **Sample inefficiency.** Genetic search requires many complete simulations and does not exploit local gradient information.

7. **Limited statistical study.** The principal portfolio results rely heavily on seed 42. Multiple independent seeds are required to quantify variance robustly.

11.3 Why the project is not NEAT

The menu permits changing the number and size of hidden layers before training, but the architecture remains fixed within each evolutionary run. Only numerical parameters evolve. NEAT, by contrast, would encode topology in the genome and permit structural mutation, such as adding nodes or connections. Implementing NEAT would therefore change the optimization problem rather than simply tune the current code.

Chapter 12

Recommended Extensions

12.1 Generalization benchmark

The highest-priority extension is not a more complex network; it is a stronger experimental protocol. A useful benchmark would define several training circuits and at least two held-out circuits. Spawn position, initial heading and perhaps sensor noise should be randomized. The final champion should then be evaluated without further evolution.

A generalization metric could be defined as

$$S_{\text{gen}} = \frac{1}{N} \sum_{j=1}^N [w_f I_j - \alpha T_j - \beta C_j], \quad (12.1)$$

where I_j indicates successful completion, T_j is race time and C_j is collision count on scenario j . This would separate memorization of a single track from reusable driving behaviour.

12.2 Multiple-seed statistical study

At least 10 independent seeds should be run for each important configuration. Results should then be reported as median and interquartile range or mean and standard deviation for quantities such as first successful generation, best race time and finish rate.

12.3 Controller-state extensions

The observation vector could be augmented with physically meaningful dynamic states,

$$\mathbf{x}_{\text{new}} = [\mathbf{x}, r/r_{\text{max}}, \beta/\beta_{\text{max}}, u_{s,k-1}, \dots], \quad (12.2)$$

or the feed-forward network could be replaced by a recurrent controller. The comparison should be performed at a fixed evaluation budget to avoid confusing network size with optimizer effort.

12.4 Optimization comparisons

The same simulator provides a good controlled benchmark for comparing:

- current genetic algorithm;

- CMA-ES or another evolution strategy;
- NEAT for topology evolution;
- a policy-gradient method such as PPO.

The comparison should use identical observations, actions, episode limits and evaluation circuits.

12.5 Vehicle-model extension

A natural mechanics-oriented extension would replace the lateral relaxation with a dynamic single-track model. With front and rear lateral tyre forces F_{yf} and F_{yr} ,

$$m(\dot{v}_y + v_x r) = F_{yf} + F_{yr}, \quad (12.3)$$

$$I_z \dot{r} = a F_{yf} - b F_{yr}, \quad (12.4)$$

and, under a linear tyre approximation,

$$F_{yf} = C_f \alpha_f, \quad (12.5)$$

$$F_{yr} = C_r \alpha_r. \quad (12.6)$$

This would create a much clearer link between the learned controller and vehicle-dynamics quantities such as understeer, lateral acceleration and tyre saturation. It should only be added if the goal shifts from an educational control environment toward a physically interpretable automotive model.

Chapter 13

Conclusion

A complete neuroevolution environment for autonomous 2D racing was implemented in Python using only NumPy for the numerical learning components and Pygame for visualization. The project combines a custom dynamic vehicle model, analytical range sensing, collision detection, curvilinear progress tracking, an explicit multilayer perceptron and a genetic optimization algorithm operating directly on network weights and biases.

The final system goes beyond a minimal demonstration by including deterministic experiment configuration, model serialization, regression tests, multiple physics versions, network-architecture experiments, fitness revisions, diagnostic visualization and direct human-versus-AI replay. The principal Sport experiment shows a clear population-level improvement over ten generations, with a large increase in successful finishers, fewer collisions and a reduction in two-lap time. A later speed-deficit objective produced a smaller further improvement.

The present result should nevertheless be interpreted as successful optimization within a defined simulator, not as generalized autonomous driving. The most scientifically valuable next step is therefore to establish multi-track and multi-seed validation before adding substantially more network complexity. With such a protocol, the project could become a compact benchmark for comparing neuroevolution, topology evolution and gradient-based reinforcement-learning methods on exactly the same controlled physical environment.

Appendix A

Current Default Parameters

Table A.1: Selected parameters from the current `Config` object.

Category	Parameter	Value
Physics	Logical time step	1/30 s
Physics	Wheelbase	24 px
Physics	Car size	30×16 px
Physics	Maximum speed	270 px/s
Physics	Engine acceleration	210 px/s ²
Physics	Braking acceleration	340 px/s ²
Physics	Max steering base	0.52 rad
Physics	Steering rate	4 rad/s
Physics	Yaw response time	0.055 s
Physics	Lateral response	32 s ⁻¹
Physics	Lateral grip limit	1100 px/s ²
Perception	Sensor angles	-90, -60, -30, -15, 0, 15, 30, 60, 90 deg
Perception	Sensor range	280 px
Network	Default hidden layers	12, 8
Network	Default hidden activation	tanh
Evolution	Default population	100
Evolution	Elite	5
Evolution	Immigrants	5
Evolution	Mutation rate	0.20 per parameter
Evolution	Mutation sigma	0.25
Episode	Maximum training time	45 s
Episode	Stall threshold	7 s
Episode	Target laps	2
Reproducibility	Default seed	42
Fitness	Speed-deficit weight	0.5

Appendix B

Suggested Final Portfolio Screenshots

The report currently uses the screenshots already contained in the project archive. For a polished final portfolio version, the following replacements would be useful. Keeping these exact file names allows the figures to be swapped without modifying the L^AT_EX source.

Suggested file	Requested content
<code>final_training.png</code>	Late generation (preferably generation 8–10), several cars visible, fitness panel and network visible.
<code>final_champion.png</code>	Final Sport-vitesse champion mid-lap on Technique fluide, with rays and dynamic network visible.
<code>final_workshop.png</code>	Network workshop showing the 10–12–8–3 architecture and selected tanh activation.
<code>final_race.png</code>	Player-versus-champion race after the countdown, both cars clearly separated on track.
<code>final_diagnostics.png</code>	Optional: stopped cars with different causes (collision/stall/time-out/finish) visible simultaneously.

Appendix C

Background References

Bibliography

- [1] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.
- [2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [3] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Pearson, 2009.
- [4] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. Springer, 2012.
- [5] G. M. Chaikin, “An algorithm for high-speed curve generation,” *Computer Graphics and Image Processing*, vol. 3, no. 4, pp. 346–349, 1974.